

ITA0606 – Machine Learning

Exp-19

Program:

Implementation of Naive Bayes Classification

for Bank Loan Prediction

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
```

Step 1: Create the dataset

```
data = {
```

```
    'Age': [25, 35, 45, 30, 50, 40, 28, 55, 32, 48,  
            27, 42, 38, 52, 29],
```

```
    'Income': [25000, 50000, 75000, 40000, 90000,  
               65000, 30000, 100000, 45000, 80000,  
               28000, 70000, 55000, 95000, 35000],
```

```
    'LoanAmount': [10000, 20000, 30000, 15000, 25000,  
                   20000, 12000, 30000, 18000, 25000,  
                   10000, 22000, 16000, 28000, 14000],
```

```
    'CreditScore': [600, 700, 750, 650, 780,
```

```
720, 620, 800, 680, 760,  
590, 730, 710, 790, 640],
```

```
'LoanApproved': [  
    'No', 'Yes', 'Yes', 'No', 'Yes',  
    'Yes', 'No', 'Yes', 'Yes', 'Yes',  
    'No', 'Yes', 'Yes', 'Yes', 'No'  
]  
}
```

```
df = pd.DataFrame(data)
```

```
# Step 2: Separate features and target
```

```
X = df[['Age', 'Income', 'LoanAmount', 'CreditScore']]  
y = df['LoanApproved']
```

```
# Step 3: Convert target labels into numbers
```

```
encoder = LabelEncoder()  
y = encoder.fit_transform(y)
```

```
# No = 0
```

```
# Yes = 1
```

```
# Step 4: Split dataset into training and testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
y,  
test_size=0.3,  
random_state=42  
)
```

```
# Step 5: Create Naive Bayes model
```

```
model = GaussianNB()
```

```
# Step 6: Train the model
```

```
model.fit(X_train, y_train)
```

```
# Step 7: Predict loan approval
```

```
y_pred = model.predict(X_test)
```

```
# Step 8: Display actual and predicted values
```

```
print("Actual Loan Status:")
```

```
print(encoder.inverse_transform(y_test))
```

```
print("\nPredicted Loan Status:")
```

```
print(encoder.inverse_transform(y_pred))
```

```
# Step 9: Calculate accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nAccuracy:")
```

```
print(accuracy * 100, "%")
```

```
# Step 10: Display confusion matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
# Step 11: Display confusion matrix details
```

```
TN, FP, FN, TP = cm.ravel()
```

```
print("\nConfusion Matrix Details:")
```

```
print("True Negative (TN):", TN)
```

```
print("False Positive (FP):", FP)
```

```
print("False Negative (FN):", FN)
```

```
print("True Positive (TP):", TP)
```

```
# Step 12: Predict loan approval for a new customer
```

```
new_customer = [[35, 60000, 20000, 720]]
```

```
prediction = model.predict(new_customer)
```

```
print("\nNew Customer Details:")

print("Age      :", new_customer[0][0])

print("Income   :", new_customer[0][1])

print("Loan Amount :", new_customer[0][2])

print("Credit Score:", new_customer[0][3])


print("\nPredicted Loan Status:")

print(encoder.inverse_transform(prediction)[0])
```

OUTPUT:

```
*** Actual Loan Status:
['Yes' 'Yes' 'No' 'Yes' 'Yes']

Predicted Loan Status:
['Yes' 'Yes' 'No' 'Yes' 'Yes']

Accuracy:
100.0 %

Confusion Matrix:
[[1 0]
 [0 4]]

Confusion Matrix Details:
True Negative (TN): 1
False Positive (FP): 0
False Negative (FN): 0
True Positive (TP): 4

New Customer Details:
Age      : 35
Income   : 60000
Loan Amount : 20000
Credit Score: 720

Predicted Loan Status:
Yes
```